



Université du Québec
à Chicoutimi

Module d'informatique et de mathématique
Département d'informatique et de mathématiques

8INF950 : Sujet spécial en cybersécurité

Cybersécurité défensive : Mapping automatique de base de
données d'incident et de détection

Date : 11 août 2026
8INF950 : Sujet spécial en cybersécurité - Été 2026

Table des matières

1	Synthèse des lectures	3
1.1	Doc 1 — 99% de faux positifs	3
1.2	Doc 2 — UCO (Unified Cybersecurity Ontology)	3
2	Introduction	4
2.1	Problématique et objectifs	4
2.2	Contexte — VERIS	4
2.3	Contexte — MITRE ATT&CK	5
2.4	Contexte — Le mapping	5
2.5	Contexte — Les 7 capability groups	6
2.6	Contexte — Versions et formats de données	6
3	Méthodologie	6
3.1	La méthode actuelle de mapping	6
3.2	État de l'art — Alarmes SIEM et modèle REACT	7
3.3	État de l'art — Ontologies et unification (UCO)	7
3.4	État de l'art — Approches IA pour l'alignement sémantique	7
3.5	Notre pipeline	7
3.5.1	Architecture du projet	9
4	Solution	9
4.1	Solution RAG	9
4.1.1	Principe et objectif	9
4.1.2	Architecture et fonctionnement	10
4.1.3	Variante attack_only	11
4.1.4	Variante with_examples	13
4.1.5	Variante???????	16
4.1.6	Variante???????	16
4.1.7	Variante???????	16
4.2	Solution Fine-tune	16
4.3	Solution PROMPT	16
4.4	Synthèse des trois solutions	16
5	Résultats	16
5.1	Configuration expérimentale	16
5.2	Résultats — Solution RAG	16
5.3	Résultats — Solution Fine-tune	16
5.4	Résultats — Solution PROMPT	16
6	Discussion	16
7	Conclusion	16

Table des figures

1	Pipeline global du projet : données brutes, trois solutions IA et comparaison aux experts.	8
---	--	---

2	Résultats de la variante <code>attack_only</code> sur la version cible VERIS 1.4.1 / ATT&CK 19.1.	13
3	Résultats de la variante <code>with_examples</code> sur la version cible VERIS 1.4.1 / ATT&CK 19.1.	15

1 Synthèse des lectures

1.1 Doc 1 — 99% de faux positifs

On distingue trois types de faux positifs (FP) :

- **Vraies alarmes** : ce sont les menaces réelles.
- **Fausse alarmes** : le système se déclenche sans raison légitime.
- **Alarmes bénignes** : alarmes techniquement correctes, mais correspondant à un comportement légitime.

La validation des alarmes repose encore sur un processus humain, grâce aux connaissances et à l'expérience des analystes.

Les auteurs proposent le modèle **REACT** : les alarmes doivent être

- **Reliable** (fiables) ;
- **Explained** (explicables) ;
- **Analytical** (analytiques) ;
- **Contextual** (contextuelles) ;
- **Transferable** (transférables à d'autres environnements).

Ce modèle est pensé pour guider la conception d'outils fondés sur l'apprentissage automatique.

1.2 Doc 2 — UCO (Unified Cybersecurity Ontology)

UCO est une ontologie sémantique visant à unifier et intégrer les données hétérogènes de cybersécurité issues de multiples standards. Son but est d'*améliorer la conscience situationnelle des analystes*.

Caractéristiques principales :

- Extension de l'ontologie IDS développée par le même groupe.
- Intégration des standards les plus utilisés en cybersécurité : STIX, CVE, CCE, CVSS, CAPEC, CybOX et la *Kill Chain*.
- Représentation en OWL/RDF plutôt qu'en XML, ce qui autorise le raisonnement automatique.
- 106 classes, 59 propriétés d'objet et 45 propriétés de données.
- Expressivité logique de niveau *ALCROIQ(D)*.

Apport. Inférence logique, mappée au *Linked Open Data* (DBpedia, YAGO, etc.), permettant le croisement des données cyber avec des connaissances générales.

Cas d'usage. Un prototype identifie les vulnérabilités liées à un type de logiciel à partir des données NVD. Au moyen de requêtes SPARQL, il recense les produits d'un éditeur et leurs vulnérabilités, afin de suggérer une alternative exempte de vulnérabilités ou de comparer l'impact sécuritaire d'un changement de fournisseur.

2 Introduction

2.1 Problématique et objectifs

Dans un centre d'opérations de sécurité (SOC), les analystes doivent constamment relier des incidents décrits dans un vocabulaire normalisé aux techniques adverses documentées dans des référentiels offensifs. Ce travail d'alignement sémantique permet d'enrichir la détection, contextualiser les alertes et améliorer la conscience situationnelle, mais il reste en grande partie manuel, coûteux et peu reproductible.

L'objectif de ce projet est de reproduire automatiquement, à l'aide de l'intelligence artificielle, le mapping entre le vocabulaire VERIS et le framework MITRE ATT&CK, puis de mesurer la qualité des mappings produits par rapport à la référence officielle publiée par le CTID (*Mappings Explorer*).

Nous avons choisi de mettre trois approches en concurrence :

- Un modèle **RAG** (Retrieval-Augmented Generation)
- Un modèle d'ia modifié pour être **fine-tuné** (fine-tuning supervisé)
- Un modèle d'ia généraliste avec **prompt engineering** (ingénierie de prompts).

Le but de notre contribution est de venir créer :

- Un pipeline reproductible permettant de générer les mappings entre VERIS et MITRE ATT&CK (préparation des données, génération, comparaison) ;
- Un format de sortie standardisé pour les mappings générés par nos solutions ;
- Des scripts de comparaison automatisée entre le mapping existant et les mappings générés par nos solutions ;

Le rapport s'organise comme suit :

- L'introduction présente le contexte technique (§2.2 à §2.6) ;
- La méthodologie décrit la pratique actuelle, l'état de l'art et notre cadre expérimental ;
- La section *Solution* détaille les trois approches retenues ;
- Les résultats, la discussion et la conclusion clôturent l'analyse.

2.2 Contexte — VERIS

VERIS (*Vocabulary for Event Recording and Incident Sharing*) est un vocabulaire structuré conçu pour décrire de manière homogène les incidents de cybersécurité. Son objectif principal est de fournir un cadre commun pour documenter ce qui s'est produit lors d'un incident, indépendamment de l'organisation ou de l'outil utilisé. Autrement dit, VERIS sert à normaliser la description des événements de sécurité afin de faciliter leur analyse, leur comparaison et leur partage entre analystes, équipes de réponse à incident et organismes de recherche.

Sur le plan de son organisation, VERIS repose sur une structure hiérarchique. Le vocabulaire est découpé en grands axes, comme **action**, **attribute**, **actor** ou encore **value_chain**, qui correspondent à différentes dimensions d'un incident. Chaque axe contient ensuite des catégories plus précises, par exemple **hacking**, **malware** ou **social** dans l'axe **action**, puis des sous-catégories et des valeurs plus fines telles que **variety** ou **vector**. Cette organisation permet de représenter un incident avec plusieurs niveaux de granularité, depuis une vue générale jusqu'à une description beaucoup plus détaillée du comportement observé.

VERIS a été développé à l'origine dans le cadre des travaux de Verizon autour du *Data Breach Investigations Report* (DBIR), avec l'idée de disposer d'un schéma cohérent pour collecter et comparer les données d'incidents à grande échelle. Avec le temps, il est devenu un standard largement reconnu dans le domaine de la réponse à incident et de l'analyse cyber, notamment parce qu'il permet d'agréger des observations provenant de sources variées tout en conservant une structure commune. Dans le cadre de ce projet, VERIS joue donc le rôle de référentiel décrivant le “quoi” de l'incident, que l'on cherche ensuite à relier au “comment” représenté par les techniques de MITRE ATT&CK.

2.3 Contexte — MITRE ATT&CK

MITRE ATT&CK (*Adversarial Tactics, Techniques, and Common Knowledge*) est une base de connaissances dédiée à la description des comportements adverses observés lors d'intrusions réelles. Contrairement à un simple catalogue de vulnérabilités ou d'outils, ATT&CK cherche à modéliser la manière dont un attaquant agit au cours d'une opération, c'est-à-dire les étapes, les objectifs intermédiaires et les techniques mobilisées. Dans cette perspective, ATT&CK décrit le “comment” d'une attaque, là où d'autres référentiels, comme VERIS, décrivent davantage le “quoi” de l'incident.

Le framework est organisé de manière hiérarchique autour de **tactiques** et de **techniques**. Les tactiques représentent les grands objectifs poursuivis par l'attaquant, par exemple l'accès initial, l'exécution, la persistance, l'élévation de privilèges ou l'impact. À l'intérieur de chaque tactique, ATT&CK recense des techniques identifiées par un code de type T1059, accompagnées d'un nom, d'une description et d'un ensemble de tactiques associées. Certaines techniques sont elles-mêmes décomposées en **sous-techniques**, notées par exemple T1059.001, ce qui permet de représenter plus finement des variantes de comportement ou des modes opératoires particuliers.

MITRE ATT&CK est maintenu par la MITRE Corporation à partir de retours d'expérience opérationnels, de rapports d'incidents et d'observations issues du renseignement sur les menaces. Il s'est progressivement imposé comme une référence majeure en cybersécurité défensive, notamment pour la détection, la chasse aux menaces, l'évaluation de couverture et la structuration des connaissances sur les attaques. Dans le cadre de ce projet, ATT&CK constitue le référentiel cible vers lequel les capacités VERIS sont mappées, afin de relier la description d'un incident aux techniques adverses susceptibles d'en expliquer le déroulement.

2.4 Contexte — Le mapping

Le mapping étudié dans ce projet consiste à relier des capacités issues du vocabulaire VERIS, qui décrivent la nature et les caractéristiques d'un incident, à des techniques du framework MITRE ATT&CK, qui décrivent les comportements et modes opératoires adverses susceptibles d'expliquer cet incident. Autrement dit, il s'agit de faire le lien entre le “quoi” observé dans VERIS et le “comment” modélisé dans ATT&CK. Concrètement, une capacité VERIS, par exemple `action.hacking.variety.SQLi`, peut être associée à une ou plusieurs techniques ATT&CK jugées pertinentes par les experts.

Ces correspondances sont produites et publiées par le CTID / MITRE Engenuity. Elles constituent dans notre travail une double référence : d'une part, elles définissent la cible que nos solutions d'intelligence artificielle cherchent à reproduire automatiquement ; d'autre part, elles servent de base d'évaluation pour mesurer la qualité des mappings

générés. Le problème traité dans ce projet n'est donc pas seulement de produire des associations plausibles entre VERIS et ATT&CK, mais bien de se rapprocher le plus possible d'un mapping expert déjà établi.

2.5 Contexte — Les 7 capability groups

Sept groupes VERIS dits « comportementaux » sont mappés vers ATT&CK. Le Tableau 1 présente les volumes pour la paire de référence VERIS 1.4.1 / ATT&CK 19.1.

Capability group	Paires VERIS↔ATT&CK	Capacités VERIS
action.hacking	538	68
action.malware	405	54
action.social	78	23
attribute.integrity	91	13
attribute.confidentiality	73	1
attribute.availability	44	6
value_chain.development	25	12

TABLEAU 1 – Les sept capability groups VERIS mappés (référence 1.4.1 / 19.1).

Cas particulier : pour `attribute.confidentiality`, les experts mappent le champ `data_disclosure` lui-même (une seule capacité), et non des valeurs sous `variety`.

2.6 Contexte — Versions et formats de données

Quatre paires de versions sont supportées de bout en bout : ATT&CK 9.0 / VERIS 1.3.5, 12.1 / 1.3.7, 16.1 / 1.4.0 et 19.1 / 1.4.1.

La **version cible d'évaluation** de ce rapport est VERIS 1.4.1 / ATT&CK 19.1 (clé `veris-1.4.1_attack-19.1-enterprise`).

Les entrées préparées pour chaque solution sont :

- `veris_<v>.json` : capacités VERIS dans les sept groupes (les « questions ») ;
- `attack_<a>.json` : techniques ATT&CK non dépréciées (les « candidats »).

Les sorties des solutions suivent le format `veris_to_mitre` ; la référence experte utilise le format CTID (`mapping_objects`).

3 Méthodologie

3.1 La méthode actuelle de mapping

Aujourd'hui, le mapping VERIS → ATT&CK est réalisé par des experts du CTID / MITRE Engenuity [?]. Le processus est essentiellement **manuel** : pour chaque capacité VERIS, l'expert analyse sa sémantique, consulte le catalogue ATT&CK et sélectionne les techniques correspondantes (y compris les sous-techniques lorsque nécessaire). Les résultats sont publiés via l'outil *Mappings Explorer* au format CTID.

Cette approche présente des **avantages** indiscutables : haute qualité, validation par des spécialistes et référence largement adoptée. Ses **limites** sont toutefois significatives : coût en temps humain, faible scalabilité et nécessité de recommencer le travail à chaque

nouvelle version de VERIS ou d'ATT&CK. Notre projet vise précisément à automatiser cette tâche tout en mesurant l'écart par rapport à cette référence experte.

3.2 État de l'art — Alarmes SIEM et modèle REACT

On distingue trois types de faux positifs (FP) dans les systèmes SIEM [?] :

- **Vraies alarmes** : menaces réelles.
- **Fausse alarmes** : déclenchement sans raison légitime.
- **Alarmes bénignes** : techniquement correctes, mais correspondant à un comportement légitime.

La validation des alarmes repose encore largement sur l'expertise humaine. Les auteurs proposent le modèle **REACT** : les alarmes doivent être **R**eliable (fiables), **E**xplained (explicables), **A**nalytical (analytiques), **C**ontextual (contextuelles) et **T**ransférable (transférables). Ce cadre guide la conception d'outils fondés sur l'apprentissage automatique. Dans notre projet, l'automatisation du mapping VERIS $\leftrightarrow$ ATT&CK vise à réduire la charge cognitive des analystes en normalisant plus rapidement le lien entre incidents et techniques adverses.

3.3 État de l'art — Ontologies et unification (UCO)

UCO (*Unified Cybersecurity Ontology*) [?] est une ontologie sémantique visant à unifier des données hétérogènes de cybersécurité (STIX, CVE, CAPEC, CybOX, *Kill Chain*, etc.) en OWL/RDF, ce qui autorise le raisonnement automatique. L'alignement VERIS $\leftrightarrow$ ATT&CK s'inscrit dans cette problématique plus large d'**unification de taxonomies cyber** : deux vocabulaires distincts doivent être reliés de manière cohérente pour améliorer la conscience situationnelle des analystes.

3.4 État de l'art — Approches IA pour l'alignement sémantique

Trois familles d'approches sont comparées dans ce projet :

- **Prompt engineering** : un LLM généraliste produit le mapping à partir d'instructions (zero-shot ou few-shot).
- **Fine-tuning** : un modèle est entraîné sur des paires expert annotées. Spécialisation forte et nous supposons une faible hallucination, mais avec un coût d'entraînement plus élevé.
- **RAG** [?] : un contexte pertinent (techniques ATT&CK, exemples experts) est récupéré avant la décision. Compromis entre interprétabilité, coût et performance.

3.5 Notre pipeline

Notre pipeline expérimental s'organise en quatre étapes principales, illustrées à la Figure 1. L'objectif est de partir de sources officielles, de produire des mappings automatisés VERIS $\rightarrow$ ATT&CK à l'aide de trois approches IA, puis de les comparer au mapping expert CTID.

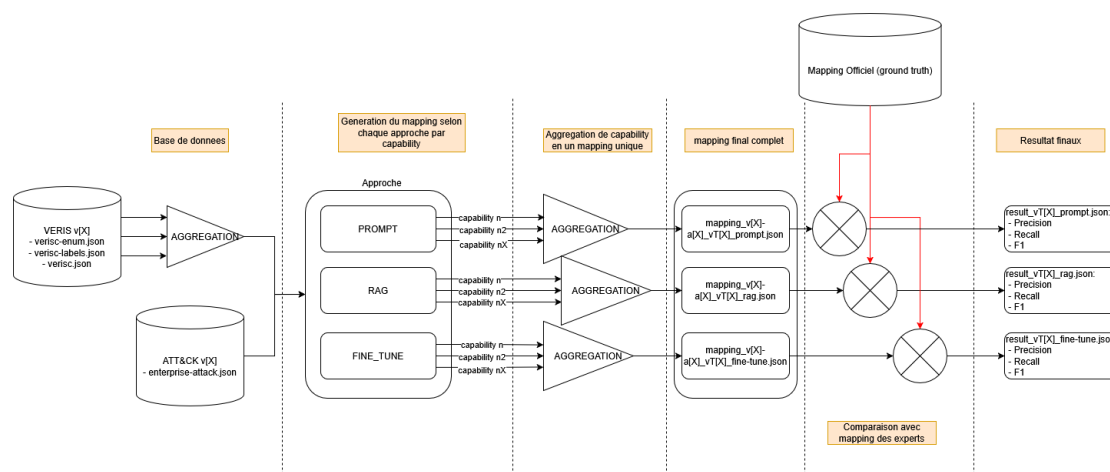


FIGURE 1 – Pipeline global du projet : données brutes, trois solutions IA et comparaison aux experts.

1. **Téléchargement des données brutes.** Les sources officielles sont rassemblées dans le dossier `data/raw/`. On y trouve les fichiers VERIS (`verisc-enum.json`, `verisc-labels.json`, `verisc.json`), le catalogue ATT&CK au format STIX (`enterprise-attack.json`) ainsi que les mappings experts publiés par le CTID (CSV, YAML, JSON ou STIX selon la version). Quatre paires de versions sont disponibles, de VERIS 1.3.5 / ATT&CK 9.0 à VERIS 1.4.1 / ATT&CK 19.1.
2. **Préparation des entrées.** Le script `tools/build_work_inputs.py` transforme ces données brutes en fichiers exploitables par les solutions IA, stockés dans `data_for_work/`. Pour chaque paire de versions, il produit :
 - `veris_<v>.json` : les capacités VERIS des sept capability groups (les « questions » à mapper) ;
 - `attack_<a>.json` : les techniques ATT&CK non dépréciées (les « candidats ») ;
 - `mapping_des_experts.json` : une copie locale du mapping de référence, utilisée pour la comparaison.
3. **Génération des mappings.** Chacune des trois solutions (`Solution_RAG`, `Solution_PROMPT`, `Solution_FINE_TUNE`) ingère les fichiers préparés et produit **sept fichiers JSON** au format `veris_to_mitre`, soit un fichier par capability group (`action.hacking`, `action.malware`, `action.social`, `attribute.integrity`, `attribute.confidentiality`, `attribute.availability`, `value_chain.development`). Les résultats sont écrits dans `Resultat/Resultat_<APPROCHE>/`.
4. **Comparaison aux experts.** Le script `Resultat/compare_veris_mappings_v2.py` confronte automatiquement les sorties des solutions au mapping expert CTID (`Resultat/Mapping_des_experts/`). Il valide la présence des sept fichiers, calcule les métriques de recouvrement (précision, rappel, F1, Jaccard) sur les paires (`veris_id`, `attack_id`) et classe les solutions entre elles.

Principe anti-fuite. Pour la version cible d'évaluation (VERIS 1.4.1 / ATT&CK 19.1), le mapping expert correspondant n'est **jamais** utilisé comme exemple d'entraînement ou d'indexation dans les solutions IA. Seules les versions antérieures (9.0, 12.1, 16.1) peuvent servir d'exemples analogiques, notamment dans la variante RAG `with_examples`.

3.5.1 Architecture du projet

Le dépôt SIEM est organisé de manière modulaire afin de séparer clairement les données, leur préparation, l'implémentation des solutions IA et l'évaluation des résultats. Cette organisation facilite la reproductibilité du pipeline et permet de traiter plusieurs paires de versions VERIS / ATT&CK dans un même cadre expérimental.

- **data/** : contient les **sources brutes** issues des dépôts officiels. Le sous-dossier **raw/attack/** regroupe les catalogues ATT&CK au format STIX, **raw/veris/** les schémas et énumérations VERIS, et **raw/mapping/** les mappings experts publiés par le CTID. Le dossier **databases/** regroupe également des bases SQLite auxiliaires utilisées pour certains vocabulaires et correspondances.
- **data_for_work/** : regroupe les **entrées préparées** pour chaque paire de versions, dans des sous-dossiers nommés selon la convention **attack-<a>_veris-<v>**. Chaque dossier contient les fichiers **veris-<v>.json** (capacités VERIS à mapper), **attack-<a>.json** (techniques ATT&CK candidates) et **mapping_des_experts.json** (copie locale de la référence experte).
- **tools/** : regroupe les **scripts de préparation et de validation** du pipeline. Le script **build_work_inputs.py** transforme les données brutes en fichiers exploitables dans **data_for_work/**, tandis que **build_example_results.py** génère des sorties de référence au format **veris_to_mitre**, utiles pour valider le format attendu.
- **Solution/** : contient le **code des trois approches IA** mises en concurrence : **Solution_RAG**, **Solution_PROMPT** et **Solution_FINE_TUNE**. Chaque solution lit les mêmes entrées préparées et produit sept fichiers JSON, un par capability group VERIS.
- **Resultat/** : centralise les **sorties expérimentales** et les outils de comparaison. On y trouve les mappings experts officiels dans **Mapping_des_experts/**, les résultats produits par chaque approche dans **Resultat_RAG/**, **Resultat_PROMPT/** et **Resultat_FINE_TUNE/**, ainsi que les scripts **compare_veris_mappings.py** et **compare_veris_mappings_v2.py** utilisés pour mesurer la qualité des mappings générés.

Cette architecture matérialise le flux décrit précédemment : les données brutes sont d'abord normalisées, puis transformées en entrées de travail, ensuite traitées par les solutions IA, et enfin comparées au mapping expert CTID.

4 Solution

Les trois solutions partagent les mêmes entrées (**veris-<v>.json**, **attack-<a>.json**) et la même sortie attendue (sept fichiers **veris_to_mitre**), mais diffèrent par leur stratégie de décision IA.

4.1 Solution RAG

4.1.1 Principe et objectif

La solution **RAG** (*Retrieval-Augmented Generation*) [?] vise à reproduire automatiquement le mapping expert entre les capacités VERIS et les techniques MITRE ATT&CK. Contrairement à une approche purement basée sur un prompt ou sur un modèle fine-tuné,

le RAG **ancrer chaque décision dans un contexte récupéré** à partir de connaissances indexées localement.

L'objectif est double :

- **réduire le risque d'hallucination**, en forçant la solution à s'appuyer sur des techniques ATT&CK réellement présentes dans le catalogue cible, plutôt que sur des identifiants inventés ;
- **exploiter une similarité sémantique** entre la description d'une capacité VERIS et les descriptions des techniques adverses.

Concrètement, pour chaque capacité VERIS, la solution construit une requête sémantique, récupère les éléments les plus pertinents dans une base vectorielle, puis produit une proposition de mapping au format `veris_to_mitre`. Dans notre expérimentation principale, le backend utilisé est `retrieval` : la décision repose sur la similarité cosinus, sans appel à un LLM. Les backends `local_llm` reste néanmoins disponible pour filtrer ou raffiner les candidats récupérés.

À partir de cette architecture commune, deux variantes sont ensuite étudiées : `attack_only` et `with_examples`. Elles partagent le même pipeline global et ne diffèrent que par le contexte injecté au moment du retrieval.

4.1.2 Architecture et fonctionnement

Cette sous-section décrit la **solution RAG globale**, c'est-à-dire le pipeline commun à toute les variantes. Les améliorations propres à chaque variante sont détaillées ensuite.

Composants communs. La solution repose sur une architecture modulaire : chargement des données VERIS et ATT&CK, calcul d'embeddings, stockage vectoriel dans ChromaDB, récupération des candidats, décision de mapping, puis validation et écriture des résultats. Les principaux modules logiciels sont présentés dans le Tableau 2.

Module	Rôle
<code>config.py</code>	Versions cible, chemins et hyperparamètres
<code>datasets.py</code>	Chargement VERIS, ATT&CK et exemples experts
<code>embeddings.py</code>	Calcul des embeddings (sentence-transformers)
<code>vectorstore.py</code>	Gestion des collections ChromaDB
<code>ingest.py</code>	Indexation des connaissances
<code>retrieve.py</code>	Récupération des candidats pertinents
<code>prompt.py</code>	Construction des prompts et du schéma JSON
<code>generator.py</code>	Backends de sélection / génération
<code>generate_mapping.py</code>	Orchestration du pipeline complet

TABLEAU 2 – Modules principaux de la solution RAG.

Indexation et base vectorielle. Dans tous les cas, le catalogue ATT&CK cible est indexé dans la collection `attack_techniques`. Chaque technique est représentée par un embedding construit à partir de son identifiant, de son nom et de sa description. La similarité utilisée est la similarité cosinus. Selon la variante, une seconde collection peut également être exploitée pour enrichir le contexte ; ce point est présenté dans les sous-sections suivantes.

Flux global de traitement. Le traitement d'une capacité VERIS suit toujours la même chaîne :

1. construction d'une requête sémantique à partir du capability group, du libellé et de la description ;
2. récupération des **top- k** techniques ATT&CK les plus similaires dans **attack_techniques** ;
3. sélection ou génération des mappings, selon le backend configuré (**retrieval** ou **local_llm**) ;
4. validation anti-hallucination : rejet des identifiants ATT&CK absents du catalogue local ;
5. enrichissement des noms, sous-techniques et tactiques à partir du catalogue ATT&CK ;
6. agrégation des entrées au format **veris_to_mitre**, regroupées en sept fichiers JSON, un par capability group.

Paramètres communs. Le paramètre principal du retrieval est **RAG_TOP_K_TECHNIQUES** (nombre de techniques candidates récupérées, 20 par défaut). Les deux variantes expérimentées partagent donc la même base technique, la même validation et le même format de sortie ; seule la nature du contexte récupéré change. C'est précisément cette différence qui est évaluée dans les sous-sections **attack_only** et **with_examples**.

Backend de décision. Deux backends de décision sont disponibles après l'étape de retrieval. Dans notre expérimentation, nous utilisons exclusivement le backend **retrieval**, qui n'appelle **aucun LLM** : les techniques candidates sont filtrées selon leur similarité cosinus avec la capacité VERIS. Concrètement, une similarité supérieure ou égale à 0,50 est considérée comme **high**, une similarité supérieure ou égale à 0,38 comme **medium** ; en dessous de ce seuil, le candidat est rejeté. Le backend **local_llm** permettrait de confier au modèle local le choix final parmi les candidats récupérés, mais il n'a pas été utilisé pour les résultats présentés dans ce rapport.

Backend de décision : Calcul de la similarité cosinus. Concrètement, chaque texte — capacité VERIS ou technique ATT&CK — est d'abord transformé en vecteur numérique par le modèle d'embeddings **sentence-transformers** (**all-MiniLM-L6-v2** en mode local), avec une normalisation de norme unitaire. La proximité entre une requête VERIS **q** et une technique ATT&CK **d** est alors mesurée par la similarité cosinus $\cos(\mathbf{q}, \mathbf{d}) = \mathbf{q} \cdot \mathbf{d} / (\|\mathbf{q}\| \|\mathbf{d}\|)$, qui se réduit au produit scalaire lorsque les vecteurs sont normalisés. ChromaDB stocke ces vecteurs dans un espace cosinus et renvoie une *distance* définie par $1 - \cos(\mathbf{q}, \mathbf{d})$; notre backend **retrieval** reconvertit ensuite cette distance en similarité ($\text{sim} = 1 - \text{distance}$) afin d'appliquer les seuils de sélection (0,50 pour **high**, 0,38 pour **medium**). Ainsi, plus deux descriptions sont sémantiquement proches, plus leur similarité cosinus est élevée, et plus la technique a de chances d'être retenue dans le mapping.

4.1.3 Variante **attack_only**

Principe et corpus. La variante **attack_only** constitue la configuration de base du RAG. Son principe est simple : pour chaque capacité VERIS, le système ne s'appuie que sur le catalogue ATT&CK cible afin de proposer des techniques correspondantes. Aucun mapping expert issu de versions antérieures n'est injecté dans le contexte.

Le corpus utilisé se limite donc à la collection vectorielle **attack_techniques**, qui contient les techniques et sous-techniques ATT&CK de la version évaluée (19.1). Chaque document indexé regroupe typiquement l'identifiant, le nom et la description de la technique. L'idée est de mesurer la qualité d'un mapping fondé uniquement sur la similarité sémantique entre le vocabulaire d'incident VERIS et le référentiel adverse ATT&CK.

Fonctionnement. Le fonctionnement reprend le pipeline global décrit précédemment, avec une restriction sur l'étape de retrieval :

1. la capacité VERIS est transformée en requête sémantique (capability group, libellé, description) ;
2. les **top- k** techniques ATT&CK les plus proches sont récupérées dans **attack_techniques** ;
3. le backend de décision retient les mappings proposés : en mode **retrieval**, aucun LLM n'est appelé ; les candidats sont filtrés uniquement selon des seuils de similarité sémantique ;
4. les identifiants hors catalogue sont rejetés, puis les libellés et tactiques sont enrichis depuis le catalogue local ;
5. le résultat est écrit au format **veris_to_mitre** dans le dossier **Resultat/Resultat_RAG/..._RAG**

Cette variante sert de **baseline** : elle permet d'évaluer ce que le RAG peut obtenir sans aide supplémentaire, avant de mesurer l'apport des exemples experts dans la variante suivante.

Résultats et remarques. Voici les résultats obtenus par la variante `attack_only` sur la version cible en sorti de notre pipeline expérimental.

```
Solution : Resultat_RAG
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_attack_only
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
  Paires experts=1250 solution=2679 communes= 193
  Precision= 7.2% Rappel= 15.4% F1= 9.8% Jaccard= 5.2%

PAR CAPABILITY_GROUP :
```

scope	exp	sol	comm	Prec	Rapp	F1
action.hacking	534	1061	72	6.8%	13.5%	9.0%
action.malware	405	1056	61	5.8%	15.1%	8.4%
action.social	79	230	26	11.3%	32.9%	16.8%
attribute.integrity	91	151	15	9.9%	16.5%	12.4%
attribute.confidentiality	73	4	1	25.0%	1.4%	2.6%
attribute.availability	44	25	3	12.0%	6.8%	8.7%
value_chain.development	25	152	15	9.9%	60.0%	16.9%

FIGURE 2 – Résultats de la variante `attack_only` sur la version cible VERIS 1.4.1 / ATT&CK 19.1.

La Figure 2 présente la sortie de `compare_veris_mappings_v2.py` pour la variante `attack_only`. Au niveau global, sur les 1 250 paires expertes de référence, la solution en propose 2 679, dont seulement 193 sont correctes. Cela se traduit par une précision de 7,2 %, un rappel de 15,4 % et un F1 de 9,8 % (Jaccard 5,2 %). Autrement dit, le système sur-génère largement des mappings : il produit plus du double de paires par rapport aux experts, mais n'en retrouve qu'une faible partie.

Le détail par capability group confirme cette tendance. Les deux scopes les plus volumineux, `action.hacking` et `action.malware`, restent proches de la moyenne globale (F1 de 9,0 % et 8,4 %), avec là aussi beaucoup plus de propositions que de paires expertes. Les meilleurs F1 s'observent sur `action.social` (16,8 %) et `value_chain.development` (16,9 %), ce dernier bénéficiant d'un rappel élevé (60 %) au prix d'une précision toujours faible. À l'inverse, `attribute.confidentiality` est le cas le plus critique : une seule paire correcte sur 73 attendues (rappel 1,4 %), malgré une précision apparente un peu meilleure (25 %) liée au très petit nombre de propositions.

Ces résultats montrent que la seule similarité sémantique VERIS ↔ ATT&CK ne suffit pas à reproduire le jugement expert. Le faible rappel indique que beaucoup de techniques pertinentes n'apparaissent pas dans le `top-k`, tandis que la faible précision révèle de nombreux faux positifs parmi les candidats proches sémantiquement. Cette baseline motive donc l'introduction d'un contexte complémentaire, via la variante `with_examples`.

4.1.4 Variante `with_examples`

Principe et corpus. La variante `with_examples` reprend l'architecture RAG globale, mais l'enrichit d'un second corpus de contexte : des mappings experts issus de versions antérieures de VERIS / ATT&CK. L'idée est de profiter d'un **transfert analogique** : si une capacité VERIS proche a déjà été mappée par des experts dans le passé, les techniques associées peuvent guider le mapping de la version cible.

Le corpus de retrieval combine donc deux collections ChromaDB :

- **attack_techniques** : le catalogue ATT&CK cible (19.1), comme dans la variante **attack_only**;
- **expert_examples** : les mappings experts des versions antérieures (ATT&CK 9.0, 12.1 et 16.1), agrégés par capacité VERIS.

Le mapping expert de la version cible (VERIS 1.4.1 / ATT&CK 19.1) n'est **jamais** indexé, afin de respecter le principe anti-fuite : les exemples servent uniquement d'analogies, jamais de fuite directe de la référence d'évaluation.

Fonctionnement. Le pipeline reste identique à celui de **attack_only**, avec une étape supplémentaire de retrieval :

1. la capacité VERIS est transformée en requête sémantique ;
2. les **top- k** techniques ATT&CK les plus proches sont récupérées dans **attack_techniques** ;
3. les **top- m** exemples experts les plus proches sont récupérés dans **expert_examples** (paramètre **RAG_TOP_M_EXAMPLES**, 5 par défaut) ;
4. le backend **retrieval** sélectionne les mappings **sans LLM**, en combinant :
 - les candidats ATT&CK dont la similarité dépasse les seuils prédéfinis ;
 - les techniques soutenues par un exemple expert proche ;
 - éventuellement quelques techniques suggérées uniquement par l'exemple le plus proche.
5. validation anti-hallucination et enrichissement depuis le catalogue local ;
6. écriture des sept fichiers JSON dans **Resultat/Resultat_RAG/..._RAG_with_examples/**.

Cette variante permet donc de mesurer l'apport concret des exemples experts par rapport à la baseline **attack_only**, à architecture et backend identiques.

Résultats et remarques. Voici les résultats obtenus par la variante `with_examples` sur la version cible en sortie de notre pipeline expérimental.

```

Solution : Resultat_RAG
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_with_examples
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
  Paires experts=1250  solution=3288  communes= 621
  Precision= 18.9%  Rappel= 49.7%  F1= 27.4%  Jaccard= 15.9%

PAR CAPABILITY_GROUP :

```

scope	exp	sol	comm	Prec	Rapp	F1
action.hacking	534	1256	223	17.8%	41.8%	24.9%
action.malware	405	1298	204	15.7%	50.4%	24.0%
action.social	79	276	50	18.1%	63.3%	28.2%
attribute.integrity	91	208	61	29.3%	67.0%	40.8%
attribute.confidentiality	73	21	18	85.7%	24.7%	38.3%
attribute.availability	44	65	41	63.1%	93.2%	75.2%
value_chain.development	25	164	24	14.6%	96.0%	25.4%

FIGURE 3 – Résultats de la variante `with_examples` sur la version cible VERIS 1.4.1 / ATT&CK 19.1.

La Figure 3 montre un gain marqué par rapport à `attack_only`. Au niveau global, sur les mêmes 1 250 paires expertes, la solution propose 3 288 mappings, dont 621 sont corrects. La précision passe à 18,9 %, le rappel à 49,7 % et le F1 à **27,4 %** (Jaccard 15,9 %). Autrement dit, l'injection d'exemples experts **plus que double le F1**, principalement grâce à une nette amélioration du rappel : près d'une paire experte sur deux est désormais retrouvée.

Le détail par capability group confirme cet apport. Les scopes volumineux `action.hacking` et `action.malware` montent respectivement à 24,9 % et 24,0 % de F1. Les meilleurs résultats s'observent sur `attribute.availability` (F1 75,2 %, rappel 93,2 %) et `attribute.integrity` (F1 40,8 %). `attribute.confidentiality` devient beaucoup plus précis (85,7 %), même si son rappel reste modéré (24,7 %). À l'inverse, `value_chain.development` atteint un rappel très élevé (96 %) mais conserve une précision faible (14,6 %), signe d'une sur-génération encore importante.

Ces résultats montrent que les exemples experts des versions antérieures apportent un contexte utile au mapping, surtout pour retrouver des paires manquées par la seule similarité ATT&CK. Le backend `retrieval` reste toutefois orienté rappel : la précision globale (18,9 %) demeure limitée, car de nombreuses propositions restent incorrectes. Un filtrage plus sélectif, par exemple via un backend LLM, constituerait une piste naturelle d'amélioration.

Et il reste à nuancer que ces résultats sont obtenus sur la version cible VERIS 1.4.1 / ATT&CK 19.1, et que l'on a injecté énormément d'exemples experts issus des versions antérieures : pour une grande partie des capacités, le RAG dispose donc déjà d'analogies proches. Il faudrait également tester l'écart entre le mapping de la version `veris-1.4.0_attack-16.1-enterprise` et celui de la version `veris-1.4.1_attack-19.1-enterprise` afin d'évaluer comment le RAG se comporte sur des cas vraiment nouveaux, absents de sa base d'exemples. Les performances pourraient alors varier selon la disponibilité et la pertinence des exemples experts.

4.1.5 Variante DeepSeek-V4-Flash-0731

Les variantes LLM qui suivent reprennent le pipeline `with_examples`, mais remplacent le backend `retrieval` (seuils de similarité) par un **LLM de sélection** interrogé via l'API Together.ai. Le `retrieval` reste inchangé : pour chaque capacité VERIS, le système fournit au modèle les `top-k` techniques ATT&CK et les `top-m` exemples experts, puis le LLM choisit parmi ces candidats uniquement (garde-fou anti-hallucination). Les trois modèles sont évalués dans les mêmes conditions (version cible VERIS 1.4.1 / ATT&CK 19.1, mode `with_examples`), afin d'isoler l'effet du choix de LLM.

Principe et corpus. Cette variante utilise le modèle `deepseek-ai/DeepSeek-V4-Flash-0731` comme backend de décision. Le principe est d'exploiter un LLM relativement rapide (« Flash ») pour filtrer et sélectionner les mappings à partir du contexte récupéré, plutôt que d'appliquer uniquement des seuils de similarité cosinus.

Le corpus reste celui de `with_examples` :

- `attack_techniques` : catalogue ATT&CK cible (19.1) ;
- `expert_examples` : mappings experts des versions antérieures (9.0, 12.1, 16.1), hors version cible.

Le mapping expert de la version évaluée n'est jamais indexé (anti-fuite). Les sorties sont déposées dans le dossier `Resultat_RAG_MultiLLM`, sous le préfixe `..._deepseek-v4-flash-0731_with_`

Fonctionnement. Pour chaque capacité VERIS :

1. construction de la requête sémantique (group, label, description) ;
2. retrieval des `top-k` techniques ATT&CK et des `top-m` exemples experts ;
3. appel à DeepSeek-V4-Flash via Together.ai (prompt système + prompt utilisateur contenant candidats et exemples) ;
4. parsing JSON de la réponse, avec extraction de secours si le modèle place le JSON dans un bloc de raisonnement ;
5. filtrage strict : seuls les `attack_id` présents dans les candidats de retrieval sont conservés ;
6. enrichissement des libellés/tactiques depuis le catalogue local et écriture au format `veris_to_mitre`.

Résultats et remarques. Voici les résultats obtenus par la variante DeepSeek-V4-Flash-0731 en sortie du script d'évaluation.

```
Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_deepseek-v4-flash-0731_with_examples
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
Paires experts=1250 solution= 378 communes= 161
Precision= 42.6% Rappel= 12.9% F1= 19.8% Jaccard= 11.0%

PAR CAPABILITY_GROUP :
scope      exp  sol  comm  Prec  Rapp  F1
action.hacking    534  102  43   42.2%  8.1%  13.5%
action.malware    405  139  48   34.5%  11.9%  17.6%
action.social      79   59   23   39.0%  29.1%  33.3%
attribute.integrity 91   38   14   36.8%  15.4%  21.7%
attribute.confidentiality 73   0    0    0.0%  0.0%  0.0%
attribute.availability 44   21   16   76.2%  36.4%  49.2%
value_chain.development 25   19   17   89.5%  68.0%  77.3%
```

FIGURE 4 – Résultats de la variante DeepSeek-V4-Flash-0731 (`with_examples`) sur VERIS 1.4.1 / ATT&CK 19.1.

La Figure ?? présente la sortie de `compare_veris_mappings_v2.py`. Au niveau global, sur les 1250 paires expertes, la solution en propose 378, dont 161 sont correctes. Cela se traduit par une précision de 42,6 %, un rappel de 12,9 % et un F1 de **19,8 %** (Jaccard 11,0 %).

Par rapport à `with_examples` en mode `retrieval` (F1 27,4 %, précision 18,9 %), DeepSeek **augmente nettement la précision** (42,6 % contre 18,9 %) mais **réduit le rappel** (12,9 % contre 49,7 %). Le LLM agit donc comme un filtre sélectif : il émet bien moins de propositions (378 contre 3288) et se trompe moins souvent, au détriment de la couverture. Le détail par capability group montre une forte hétérogénéité. `value_chain.development` atteint un F1 de 77,3 % (précision 89,5 %, rappel 68,0 %), et `attribute.availability` un F1 de 49,2 %. À l'inverse, `attribute.confidentiality` reste à 0 % (aucune paire émise), et les scopes volumineux `action.hacking` et `action.malware` restent modestes (F1 de 13,5 % et 17,6 %). Globalement, DeepSeek illustre bien le compromis précision / rappel lorsque le backend de décision est un LLM plutôt qu'un seuil de similarité.

4.1.6 Variante Llama-3.3-70B-Instruct-Turbo

Principe et corpus. Cette variante utilise le modèle `meta-llama/Llama-3.3-70B-Instruct-Turbo` comme backend de décision. Il s'agit du plus grand des trois modèles retenus (70 milliards de paramètres, variante Turbo serverless sur Together.ai), censé offrir le meilleur suivi d'instructions pour le filtrage des candidats ATT&CK.

Le corpus de retrieval est identique aux autres variantes multi-LLM (`attack_techniques` + `expert_examples`, hors version cible). Les sorties sont écrites dans `Resultat_RAG_MultiLLM`, sous le préfixe `..._llama-3-3-70b-instruct-turbo_with_examples`.

Fonctionnement. Le pipeline est le même que pour DeepSeek : `retrieval with_examples`, puis appel chat au modèle Llama via l'API Together, parsing JSON, filtre strict sur les `attack_id` candidats, et écriture des sept fichiers de scope. Seul change l'identifiant du modèle passé à l'API ; prompts, k , m et post-traitement restent figés pour permettre une comparaison équitable.

Résultats et remarques. Voici les résultats obtenus par la variante Llama-3.3-70B-Instruct-Turbo en sortie du script d'évaluation.

```
Solution : Resultat_RAG_MultiLLM
Dossier : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_llama-3-3-70b-instruct-turbo_with_examples
Version : veris-1.4.1_attack-19.1-enterprise
Experts : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
  Paires experts=1250 solution= 404 communes= 192
  Precision= 47.5% Rappel= 15.4% F1= 23.2% Jaccard= 13.1%

PAR CAPABILITY_GROUP :
```

scope	exp	sol	comm	Prec	Rapp	F1
action.hacking	534	124	62	50.0%	11.6%	18.8%
action.malware	405	139	49	35.3%	12.1%	18.0%
action.social	79	49	23	46.9%	29.1%	35.9%
attribute.integrity	91	42	19	45.2%	20.9%	28.6%
attribute.confidentiality	73	4	4	100.0%	5.5%	10.4%
attribute.availability	44	16	15	93.8%	34.1%	50.0%
value_chain.development	25	30	20	66.7%	80.0%	72.7%

FIGURE 5 – Résultats de la variante Llama-3.3-70B-Instruct-Turbo (with_examples) sur VERIS 1.4.1 / ATT&CK 19.1.

La Figure ?? montre les meilleurs scores parmi les trois backends LLM. Globalement, la solution propose 404 paires contre 1250 expertes, dont 192 correctes : précision 47,5 %, rappel 15,4 %, F1 de **23,2 %** (Jaccard 13,1 %).

Llama combine donc une précision encore un peu supérieure à DeepSeek (47,5 % contre 42,6 %) et un rappel légèrement meilleur (15,4 % contre 12,9 %). Le F1 global (23,2 %) dépasse DeepSeek (19,8 %) et Qwen (14,5 %), tout en restant sous le F1 du backend retrieval with_examples (27,4 %), dominé par un rappel bien plus élevé.

Par capability group, les points forts sont value_chain.development (F1 72,7 %, rappel 80 %) et attribute.availability (F1 50,0 %, précision 93,8 %). Contrairement à DeepSeek, Llama produit au moins quelques paires correctes sur attribute.confidentiality (4/4 propositions justes, mais rappel seulement 5,5 %). Les scopes action.hacking et action.malware restent limités en rappel (11,6 % et 12,1 %), ce qui tire le score global vers le bas malgré une précision autour de 35–50 %. Llama apparaît donc comme le meilleur filtre LLM de notre panel, sans pour autant retrouver le volume de paires correctes de la baseline retrieval orientée rappel.

4.1.7 Variante Qwen2.5-7B-Instruct-Turbo

Principe et corpus. Cette variante utilise le modèle Qwen/Qwen2.5-7B-Instruct-Turbo comme backend de décision. Il s'agit d'un modèle nettement plus petit (7 milliards de paramètres) que Llama-70B, retenu pour comparer une famille distincte (Alibaba Qwen) en accès serverless, avec un coût et une latence plus bas.

Le corpus reste celui de with_examples (attack_techniques + expert_examples). Les sorties sont déposées dans Resultat_RAG_MultiLLM, sous le préfixe ..._qwen2-5-7b-instruct-turbo_

Fonctionnement. Le fonctionnement est strictement parallèle aux deux variantes précédentes : même retrieval, mêmes prompts, même garde-fou sur les identifiants candidats. Seul le modèle Qwen/Qwen2.5-7B-Instruct-Turbo change, ce qui permet d'isoler l'impact de la taille et de la famille de modèle sur la qualité du mapping.

Résultats et remarques. Voici les résultats obtenus par la variante Qwen2.5-7B-Instruct-Turbo en sortie du script d'évaluation.

```
Solution : Resultat_RAG_MultiLLM
Dossier  : veris-1.4.1_attack-19.1-enterprise_RAG_MultiLLM_qwen2-5-7b-instruct-turbo_with_examples
Version  : veris-1.4.1_attack-19.1-enterprise
Experts  : veris-1.4.1_attack-19.1-enterprise_json.json
Fichiers : 7/7 scopes presents

GLOBAL (tous scopes agregés) :
Paires experts=1250 solution= 203 communes= 105
Precision= 51.7% Rappel= 8.4% F1= 14.5% Jaccard= 7.8%

PAR CAPABILITY_GROUP :
```

scope	exp	sol	comm	Prec	Rapp	F1
action.hacking	534	73	38	52.1%	7.1%	12.5%
action.malware	405	69	27	39.1%	6.7%	11.4%
action.social	79	17	9	52.9%	11.4%	18.7%
attribute.integrity	91	17	9	52.9%	9.9%	16.7%
attribute.confidentiality	73	2	2	100.0%	2.7%	5.3%
attribute.availability	44	9	7	77.8%	15.9%	26.4%
value_chain.development	25	16	13	81.2%	52.0%	63.4%

FIGURE 6 – Résultats de la variante Qwen2.5-7B-Instruct-Turbo (with_examples) sur VERIS 1.4.1 / ATT&CK 19.1.

La Figure ?? montre le profil le plus *conservateur* des trois LLM. Sur les 1 250 paires expertes, la solution n'en propose que 203, dont 105 correctes : précision **51,7 %** (la meilleure des trois LLM), rappel 8,4 % et F1 de **14,5 %** (Jaccard 7,8 %).

Qwen maximise donc la confiance des paires émises, au prix d'un rappel très bas : moins d'une paire experte sur douze est retrouvée. Côté scopes, `value_chain.development` reste le meilleur cas (F1 63,4 %), et `attribute.confidentiality` affiche encore 100 % de précision mais sur seulement 2 propositions (rappel 2,7 %). Les scopes `action.hacking` et `action.malware` chutent à 12,5 % et 11,4 % de F1, avec un rappel autour de 7 %.

En synthèse des trois backends LLM : Llama-3.3-70B obtient le meilleur F1 (23,2 %), DeepSeek-V4-Flash un profil intermédiaire (19,8 %), et Qwen2.5-7B la meilleure précision (51,7 %) mais le F1 le plus bas (14,5 %). Tous trois privilégient la précision par rapport au backend `retrieval` de `with_examples`, qui reste supérieur en F1 grâce à son fort rappel. Le choix du LLM apparaît donc comme un régulateur du compromis précision/rappel plutôt que comme un simple gain uniforme.

4.2 Solution Fine-tune

4.3 Solution PROMPT

4.4 Synthèse des trois solutions

5 Résultats

5.1 Configuration expérimentale

5.2 Résultats — Solution RAG

5.3 Résultats — Solution Fine-tune

5.4 Résultats — Solution PROMPT

6 Discussion

7 Conclusion